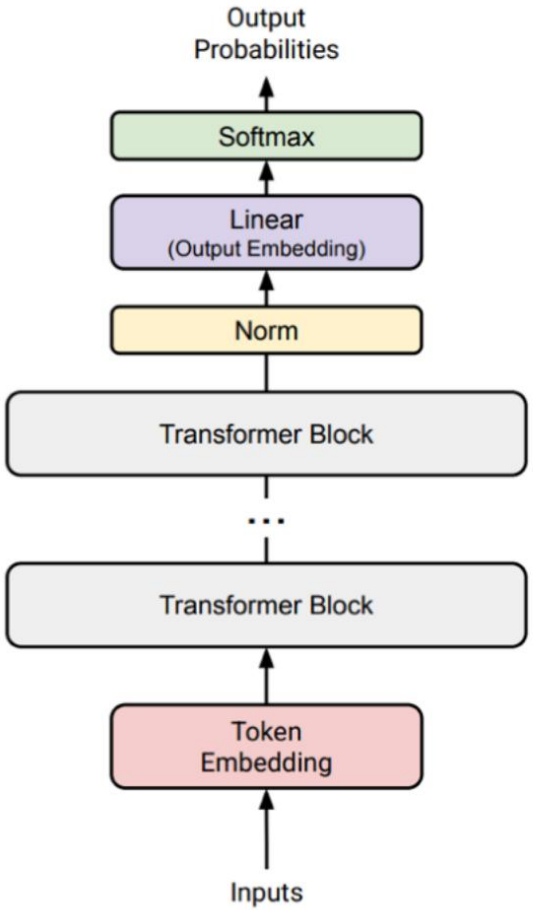
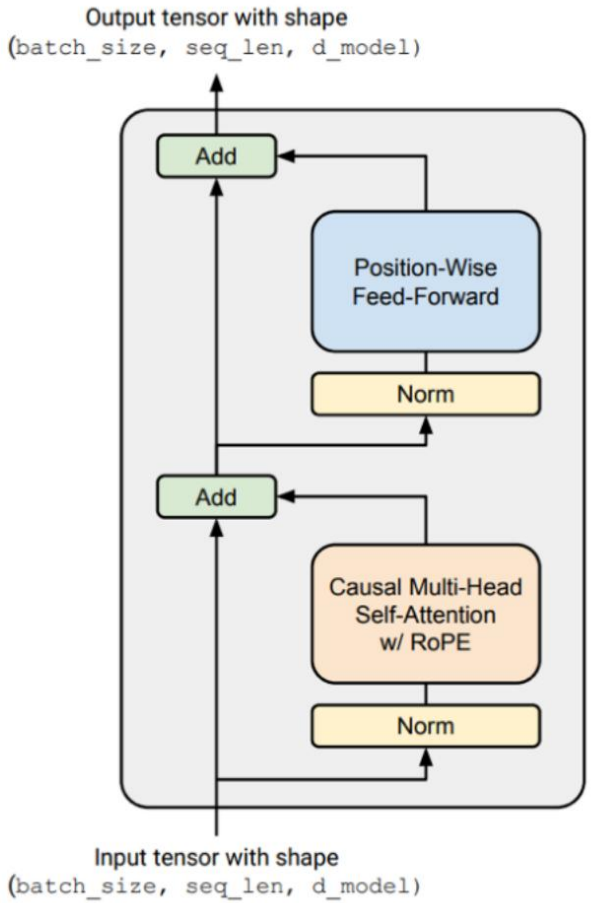
Modern Transformer design choices

Stable blocks, position information, efficient attention and experts

Parallel Programming for LLMs
Lecture 4

From the basic decoder to a modern block





Stable activations

Choose normalization and its position in the residual block.

Position information

Let attention distinguish token order and distance.

Feedforward computation

Use gating to control which features pass through.

Layer normalization

Normalize each token across its hidden dimensions.

Center using the mean μ, then rescale using the variance σ².

$$\boxed{\mathrm{LN}(x) = \gamma\hat{x} + \beta}$$

Learned γ and β adjust the scale and offset. ε prevents division by zero.

$$\text{with} \quad \hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

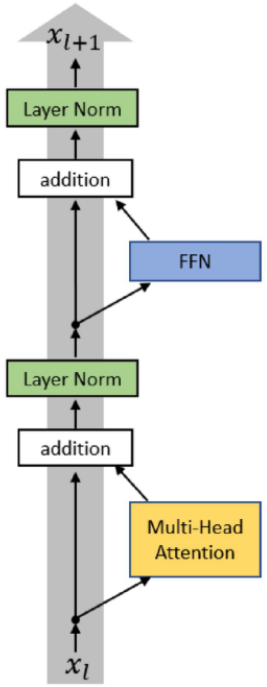
$$\text{where} \quad \mu = \frac{1}{d}\sum_{i=1}^{d} x_i \quad \text{and} \quad \sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$$

Ba et al., 2016. Layer Normalization

Pre-norm and post-norm

Post-norm

$$\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$$



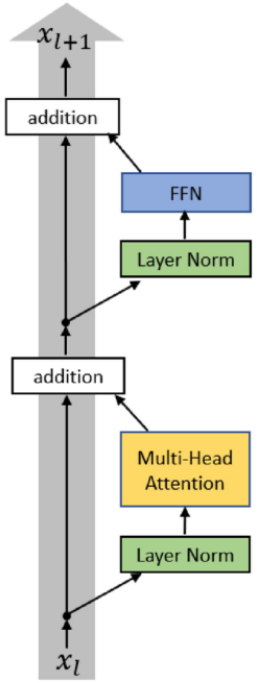
After

residual connection

Normalize after adding the residual.

Pre-norm

$$\text{Output} = x + \text{SubLayer}(\text{LayerNorm}(x))$$



Before

residual connection

Keep a direct residual path.

Xiong et al., 2020. On Layer Normalization in the Transformer Architecture

RMSNorm

Rescale the hidden vector without subtracting its mean.

$$\mathrm{RMS}(x) = \sqrt{[(1/d)\sum_i x_i^2 + \varepsilon]}$$

$$y_i = \gamma_i x_i \,/\, \mathrm{RMS}(x)$$

d is the hidden dimension. γ is a learned scale for each coordinate.

Compared with LayerNorm, RMSNorm omits mean centering and usually the additive bias.

Zhang and Sennrich, 2019. Root Mean Square Layer Normalization

Normalization and data movement

Few arithmetic operations can still take a noticeable share of runtime.

Operator class	FLOPs (%)	Runtime (%)
Tensor contraction	99.80	61.0
Normalization	0.17	25.5
Element-wise	0.03	13.5

Normalization and element-wise operations repeatedly read and write activations.

These measurements illustrate one workload. The percentages depend on the model and hardware.

Ivanov et al., 2023

Gated feedforward layers

An ordinary FFN expands the hidden vector, applies a nonlinearity and projects back.

$$\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1)\, W_2$$

A gated FFN multiplies two projections element by element before the output projection.

$$\mathrm{ReGLU}(x) = (\mathrm{ReLU}(xW_1) \odot xV)\, W_2$$

$\odot$ is the element-wise product. The extra projection modulates which features pass through.

Shazeer, 2020. GLU Variants Improve Transformer

GeGLU and SwiGLU

GeGLU uses GELU for the gate.

$$\mathrm{GeGLU}(x) = (\mathrm{GELU}(xW_1) \odot xV)\, W_2$$

SwiGLU uses SiLU(x) = x · sigmoid(x).

$$\mathrm{SwiGLU}(x) = (\mathrm{SiLU}(xW_1) \odot xV)\, W_2$$

Both use three weight matrices. The next slide accounts for the extra parameters.

Shazeer, 2020. GLU Variants Improve Transformer

Feedforward width and parameter count

Let d be the model dimension and f the FFN hidden width.

Ordinary FFN

Two matrices: about 2df parameters.
A common width f = 4d gives $8d^2$.

Gated FFN

Three matrices: about 3df parameters.
A width $f \approx 8d/3$ also gives $8d^2$.

This comparison ignores biases. Implementations often round f to hardware-friendly multiples.

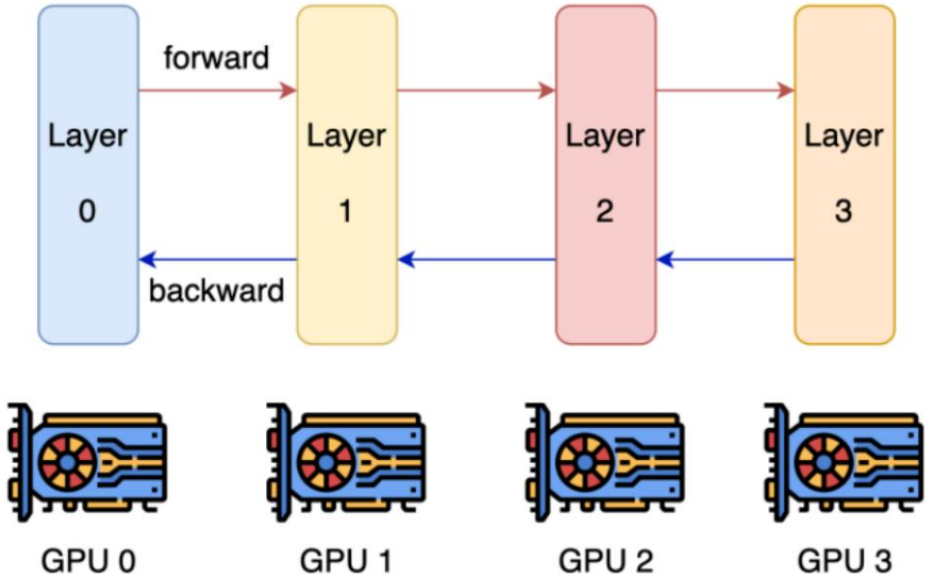
Shazeer, 2020

Depth, width and parallelism

Depth creates sequential dependencies between layers.

Width allows matrix operations to be split across devices.

Pipeline parallelism places groups of layers on different devices.



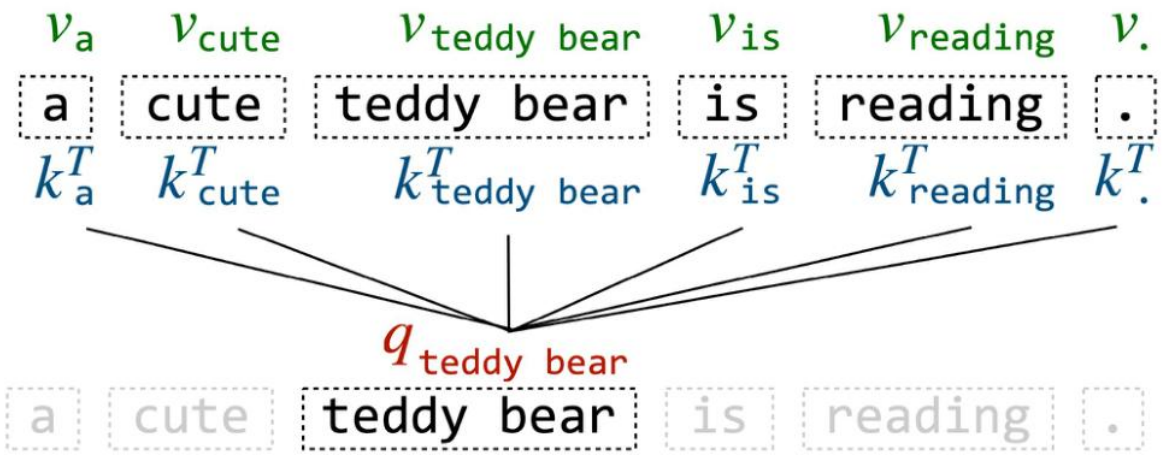
Tay et al., 2021

Position information

How does attention know where a token is?

Why position information matters

Content-based attention alone does not identify a token's position.



For a fixed query, reordering content-only key/value pairs leaves the weighted sum unchanged.

Position signals let the model distinguish word order and distance.

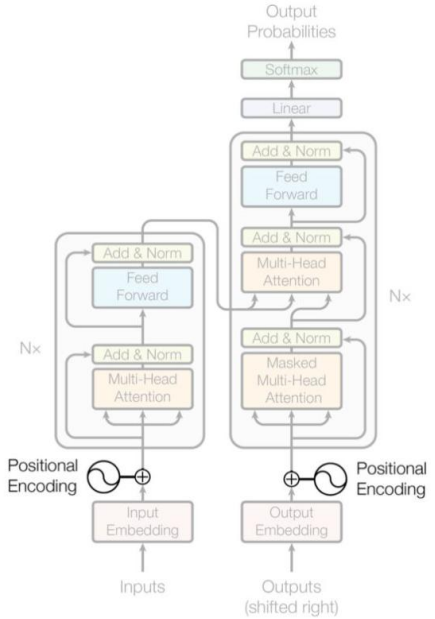
Learned absolute positions

Add a learned position vector to the token embedding.

$$h_m = \text{token_embedding}(x_m) + \text{position_embedding}(m)$$

Each position has its own trainable vector.

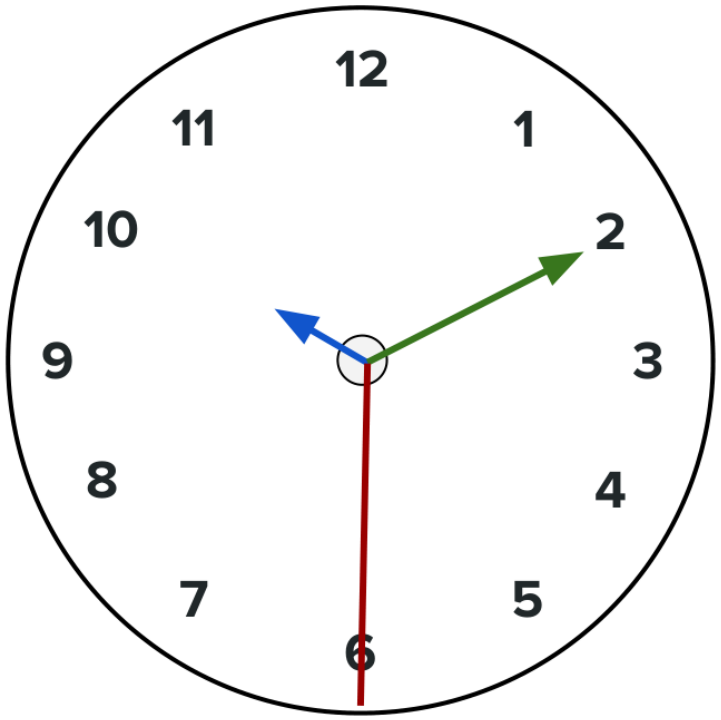
Positions outside the learned table need an extension strategy.



Vaswani et al., 2017; Amidi and Amidi, Lecture 2

Position signals at different frequencies

A clock combines hands that rotate at different speeds.



Hour (slow)

Minute (fast)

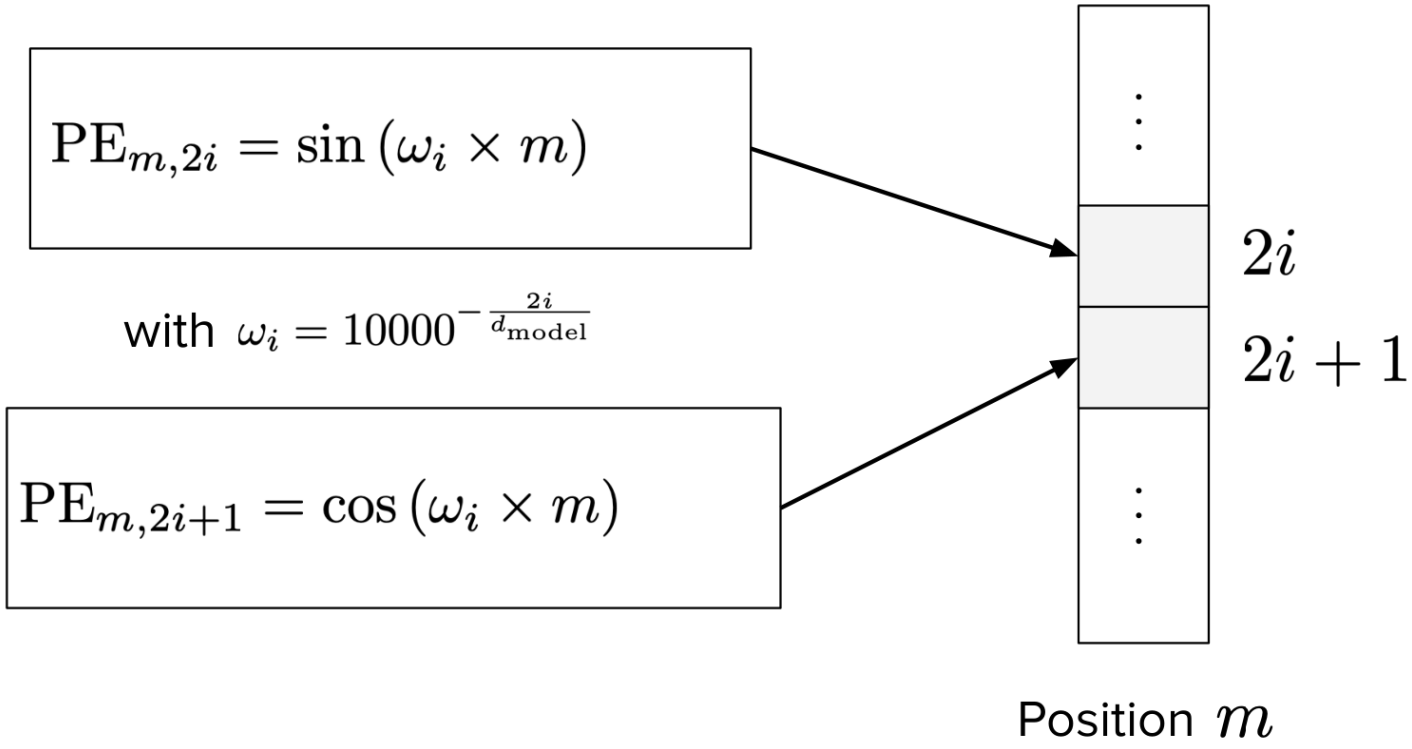
Second (very fast)

Sinusoidal positions use the same idea: coordinate pairs vary at different frequencies.

Together, the pairs encode the token's position.

Sinusoidal position embeddings

Each coordinate pair uses one frequency ω_i.



m is the token position. d_model is the embedding width.

Vaswani et al., 2017. Attention Is All You Need

Relative position in attention

An attention score should reflect both content and the distance between tokens.

Absolute positions

Query at m = 12
Key at n = 9

Relative offset

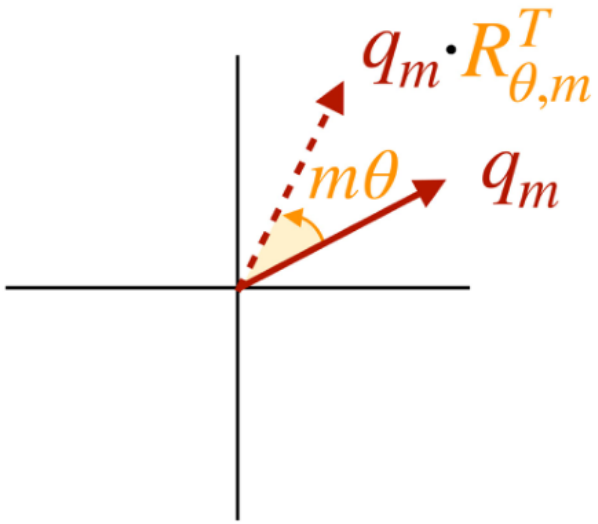
$$n - m = -3$$

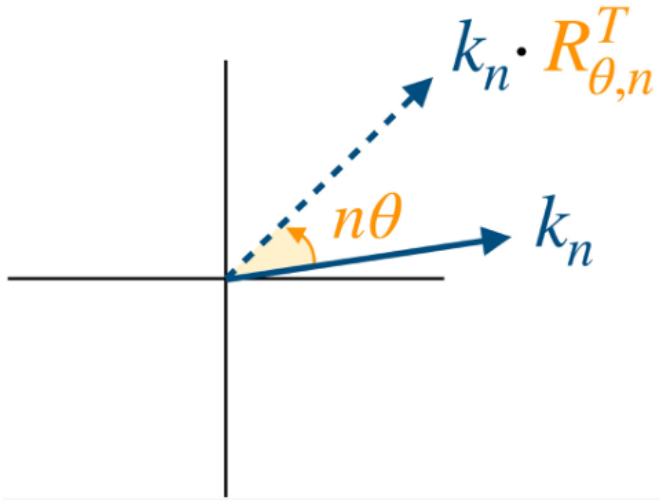
Shift both positions by the same amount and the relative offset stays unchanged.

Su et al., 2021. RoFormer

RoPE: rotating queries and keys

Rotate each query and key according to its token position.





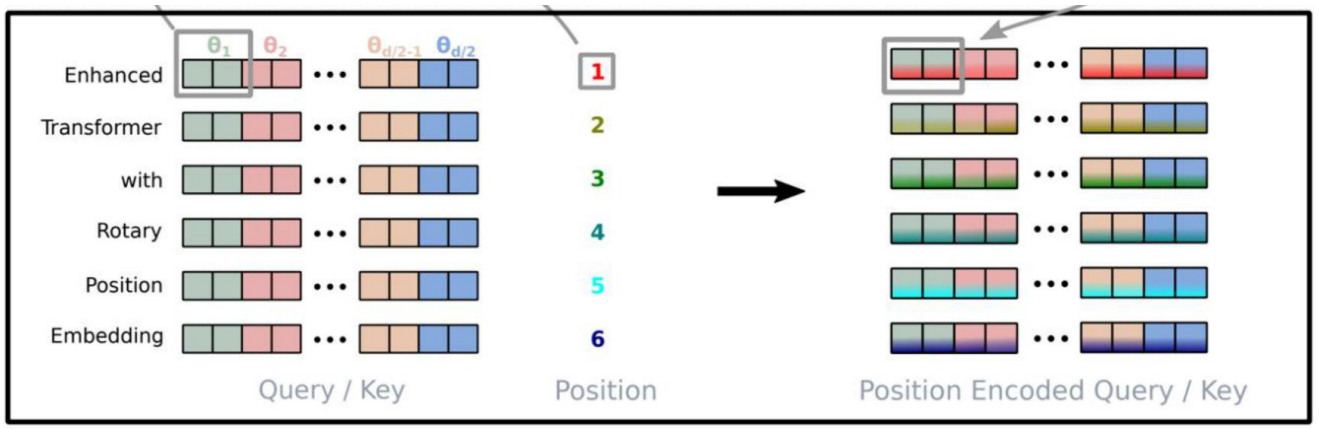
One coordinate pair uses a 2 × 2 rotation matrix:

$$R_{\theta,m} = \begin{pmatrix} \cos(m\theta) & -\sin(m\theta) \\ \sin(m\theta) & \cos(m\theta) \end{pmatrix}$$

Su et al., 2021. RoFormer; Amidi, 2024

RoPE: coordinate pairs and relative offsets

Rotate every pair of coordinates, with a different frequency for each pair.



For column vectors:

$$(R_m q)^T (R_n k) = q^T R_{n-m} k$$

The positional dependence is the relative offset n − m.

Su et al., 2021. RoFormer

RoPE: the full rotation matrix

Each diagonal block rotates one coordinate pair.

$$f_{\{q,k\}}(\boldsymbol{x}_m, m) = \boldsymbol{R}^d_{\Theta,m} \boldsymbol{W}_{\{q,k\}} \boldsymbol{x}_m \tag{14}$$

$$\boldsymbol{R}^d_{\Theta,m} = \begin{pmatrix} \cos m\theta_1 & -\sin m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ \sin m\theta_1 & \cos m\theta_1 & 0 & 0 & \cdots & 0 & 0 \\ 0 & 0 & \cos m\theta_2 & -\sin m\theta_2 & \cdots & 0 & 0 \\ 0 & 0 & \sin m\theta_2 & \cos m\theta_2 & \cdots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & \cos m\theta_{d/2} & -\sin m\theta_{d/2} \\ 0 & 0 & 0 & 0 & \cdots & \sin m\theta_{d/2} & \cos m\theta_{d/2} \end{pmatrix} \tag{15}$$

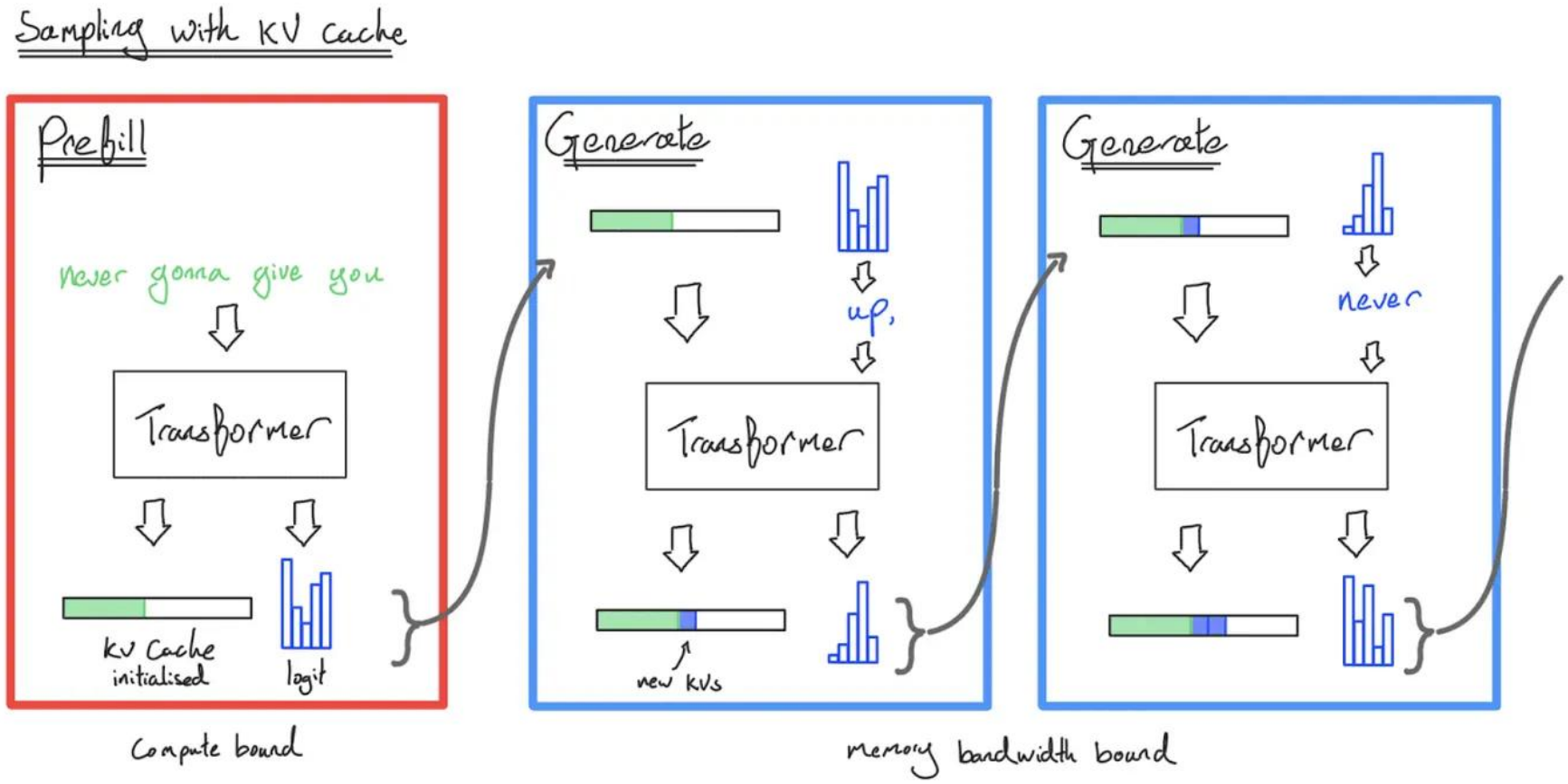
m is the token position; θ_i is the rotation frequency for coordinate pair i.

Attention efficiency

What grows when the context gets longer?

Autoregressive decoding and the KV cache

Prefill initializes the cache. Each decoding step reuses past K/V and appends the new token's K/V.



How to Scale Your Model: inference, JAX scaling book

KV-cache memory

For a standard attention cache:

$$\text{bytes} \approx 2 \cdot B \cdot T \cdot L \cdot G \cdot d_h \cdot b$$

B: batch size T: cached tokens L: layers

G: KV heads d_h: head dimension b: bytes per element

The factor 2 stores both keys and values. Fewer KV heads reduce cache memory and reads.

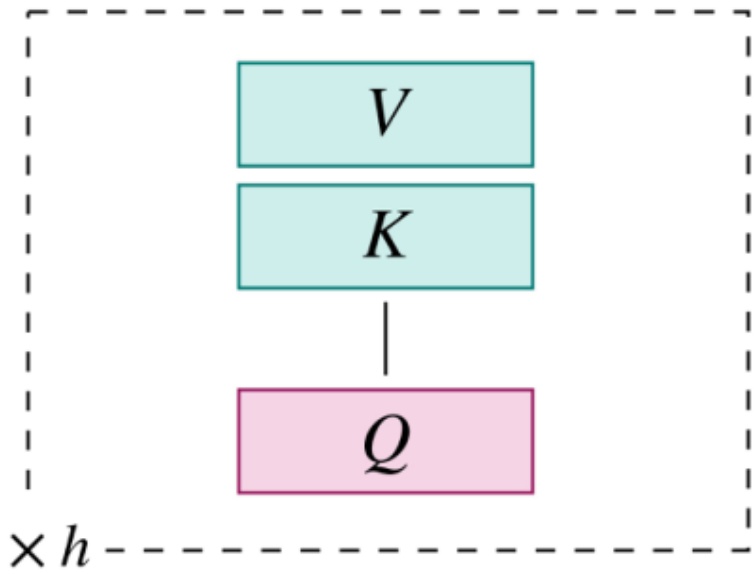
Course lecture 10, KV-cache size calculation

Multi-head attention: separate K/V heads

Each query head has its own key and value projections.

$$G = h$$

h query heads
h key heads
h value heads



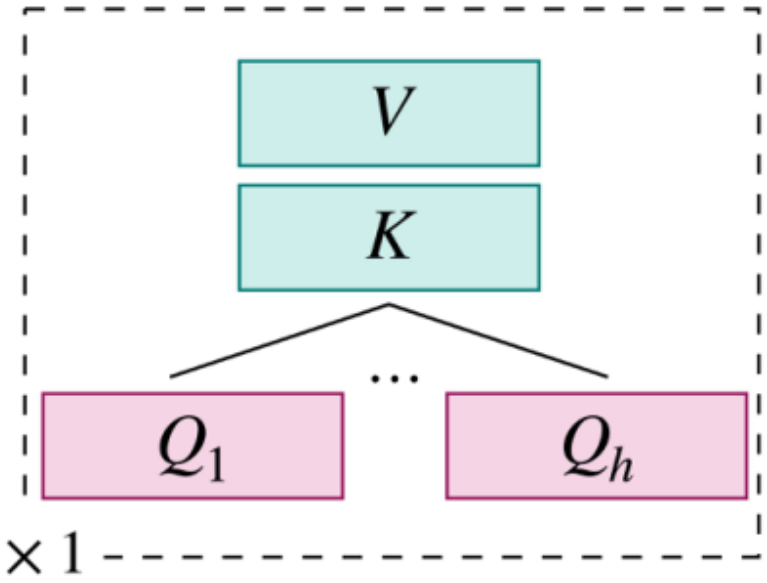
The cache stores K/V for every head.

Multi-query attention: one shared K/V head

Keep multiple query heads, but share one key and one value head.

$$G = 1$$

For fixed head dimension, the cache uses about 1/h of the MHA K/V storage.



Sharing reduces memory traffic, with a possible quality trade-off.

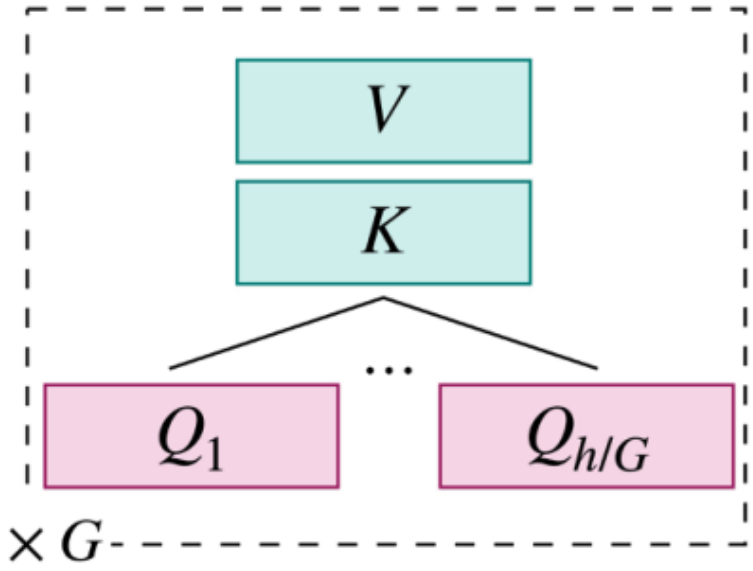
Shazeer, 2019. Fast Transformer Decoding; Amidi, 2024

Grouped-query attention

Each group of query heads shares a key and value head.

$$1 < G < h$$

Each K/V head serves h/G query heads.
Cache storage is G/h of MHA.



GQA offers an intermediate choice between MHA and MQA.

Ainslie et al., 2023. GQA; Amidi, 2024

A KV-cache example

One sequence, 4,096 tokens, 32 layers, 32 query heads.

Each head has 128 features. Each cached value uses 2 bytes.

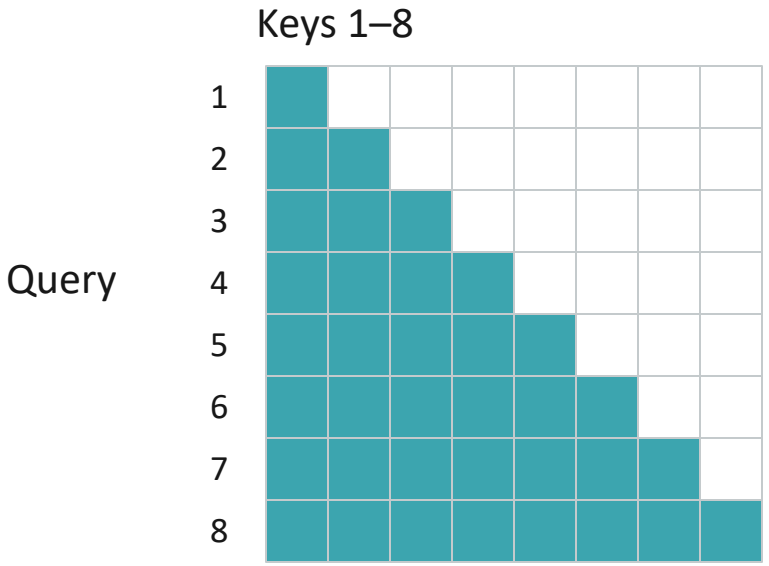
Attention	K/V heads G	Cache memory
MHA	32	2 GiB
GQA	8	512 MiB
MQA	1	64 MiB

Changing only the number of K/V heads changes the cache size.

Sliding-window attention

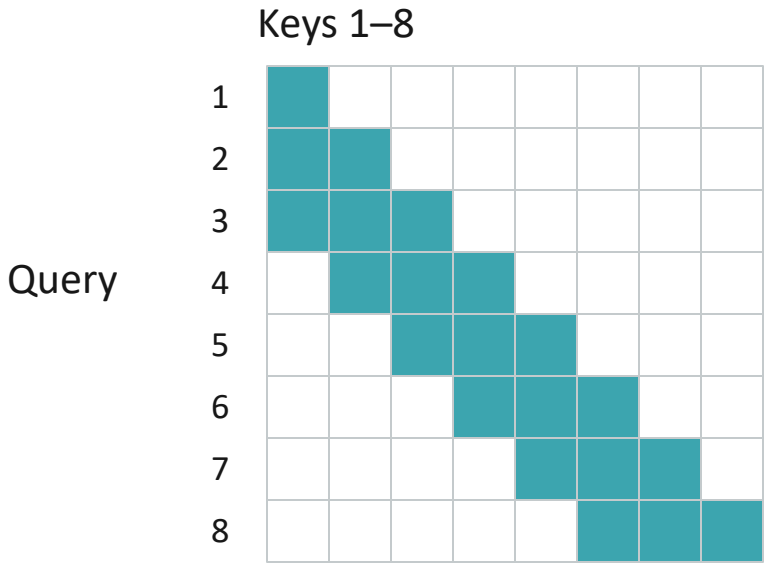
A causal window keeps only the most recent w keys for each query.

Full causal attention



Every earlier token is directly available.

Window w = 3



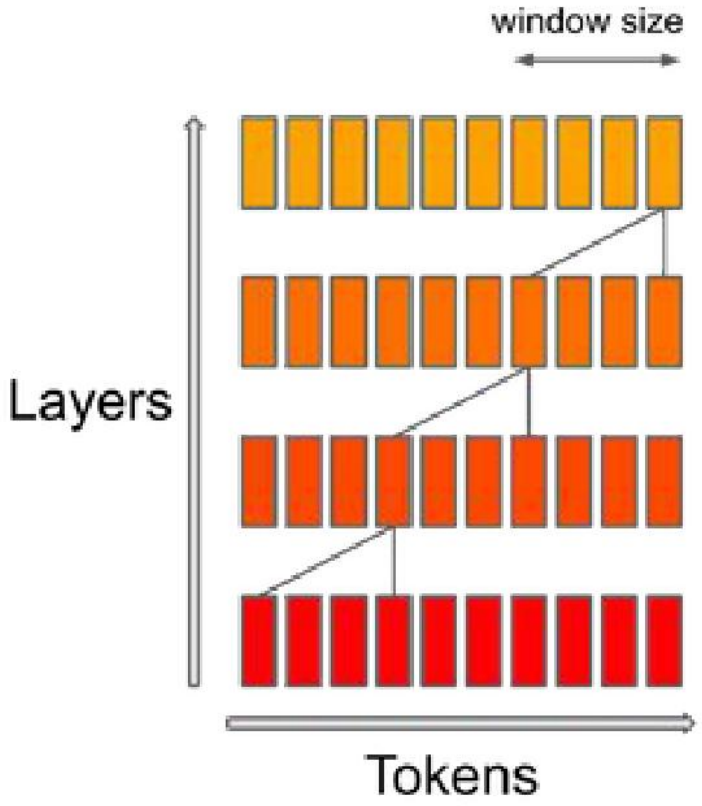
Each token reads at most 3 positions.

Attention-score work changes from $O(T^2)$ to $O(Tw)$.

Adapted from CME 295 Lecture 2, local attention

A wider receptive field across layers

Local attention passes information across successive layers.



Deeper layers can depend on tokens outside one layer’s direct window.

Some architectures mix local layers with global attention to retain longer-range access.

Jiang et al., 2023. Mistral 7B

Linear attention: associativity

Consider the usual attention operation.. $Q \in R^{n \times d_k}, K \in R^{n \times d_k}, V \in R^{n \times d_v}$

$$Attn(Q, K, V) = \rho(QK^\top)V$$

With ρ = identity (no softmax), matrix multiplication can be reordered.

$$(QK^\top)V = Q(K^\top V)$$

This avoids the n × n attention matrix: cost is linear in n for fixed head dimensions.

Shen et al. (2018); Katharopoulos et al. (2020)

Linear attention as a recurrence

Recall that in purely linear attention, we consider the reordering

$$(QK^\top)V = Q(K^\top V)$$

For causal attention, maintain a state that sums key–value outer products.

$$S_t = S_{t-1} + k_t v_t^\top \quad \text{and} \quad y_t = q_t^\top S_t$$

Training can use parallel scans or chunks. Decoding updates one state per token.

The state size depends on the head dimensions, not the sequence length.

Three ways to reduce attention cost

Each method changes a different part of attention.

Method	What gets smaller	Main constraint
MQA / GQA	Stored K/V heads	Less head-specific K/V
Local / sparse attention	Direct token connections	Limited direct context
Linear attention	History state	Changes the attention rule

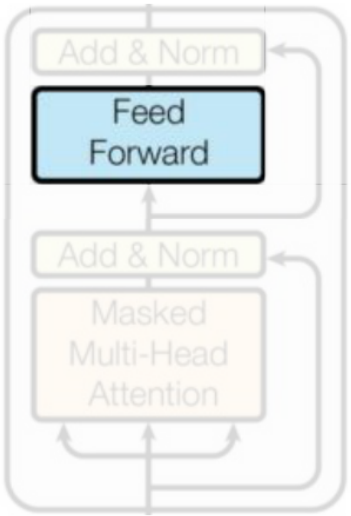
These choices can be combined, but their quality and speed depend on the model and hardware.

Mixtures of experts

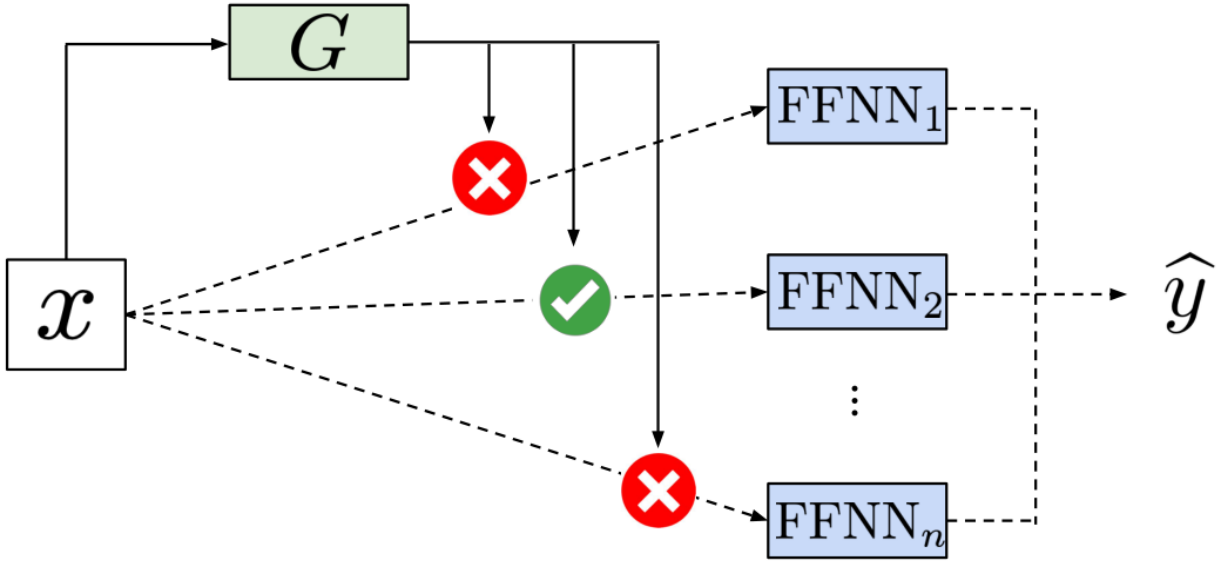
Can we add parameters without running them all for every token?

MoE inside a Transformer

Expert FFNs replace selected dense feedforward layers.



Nx

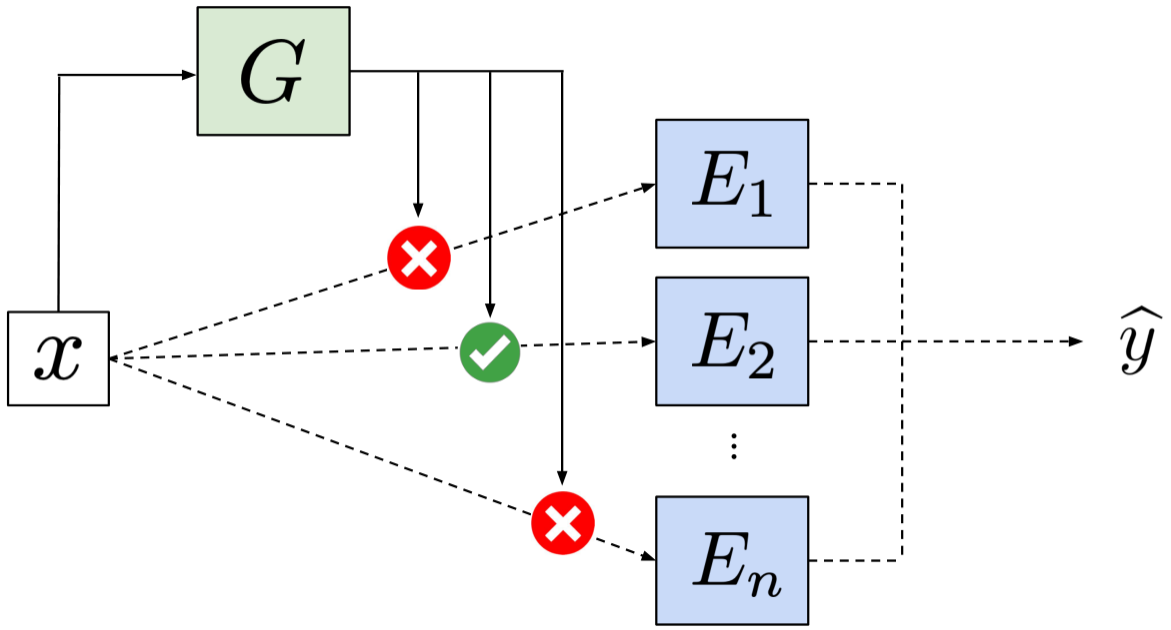


Routing happens separately for each token. Attention still mixes information across tokens.

Vaswani et al., 2017; Shazeer et al., 2017

Mixtures of experts

A router chooses a small subset of expert networks for each token.



More expert parameters can increase capacity without running every expert on every token.

Shazeer et al., 2017. Sparsely-Gated Mixture-of-Experts Layer

A token routed to two experts

Four experts receive these router probabilities. Choose k = 2.

E_1	E_2	E_3	E_4
0.10	**0.60**	0.05	**0.25**

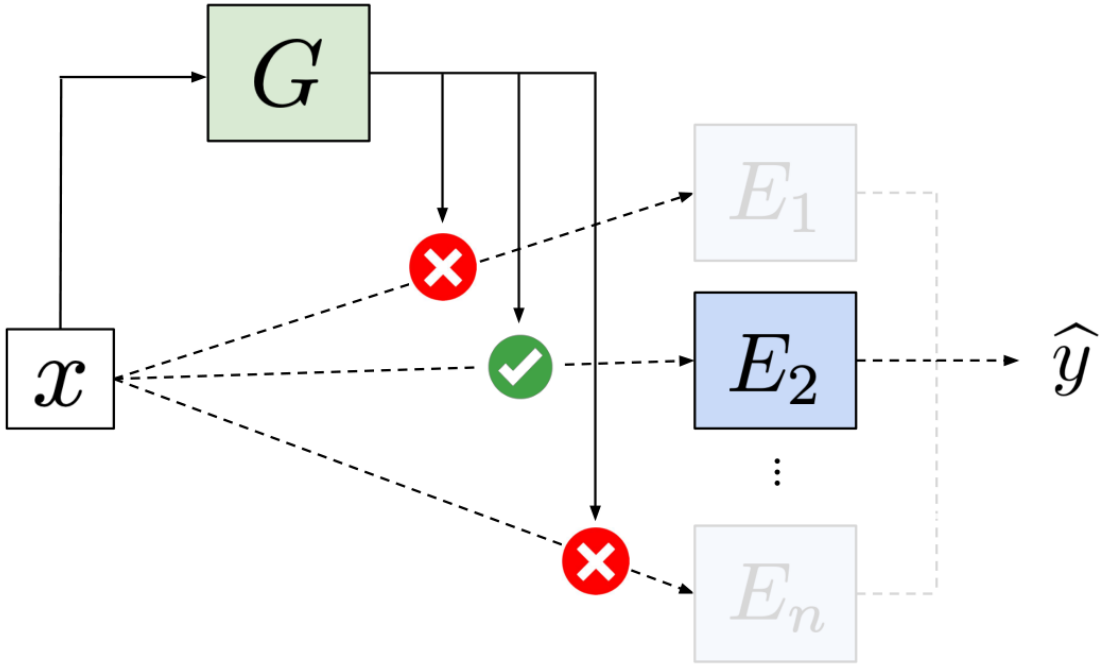
E_2 and E_4 receive the token. Their probabilities sum to 0.85.

Renormalized weights: 0.60 / 0.85 ≈ 0.706 and 0.25 / 0.85 ≈ 0.294.

$$\text{Output} \approx 0.706\ \mathrm{E}_2(\mathrm{x}) + 0.294\ \mathrm{E}_4(\mathrm{x})$$

Top-k expert routing

Score the experts, select k, then combine the selected outputs.



$$\hat{y} = \sum_{i \in \mathcal{I}_k} G(x)_i E_i(x)$$

I_k: selected experts
$G(x)_i$: routing weight
$E_i(x)$: expert output

The router learns which experts each token uses.

Shazeer et al., 2017. Sparsely-Gated Mixture-of-Experts Layer

Top-k routing with normalized weights

1. Score the experts

$$p = \mathrm{softmax}(xW_r)$$

W_r is the learned router projection.

2. Select the k largest probabilities

$$I_k = \mathrm{TopK}(p)$$

3. Normalize the selected weights

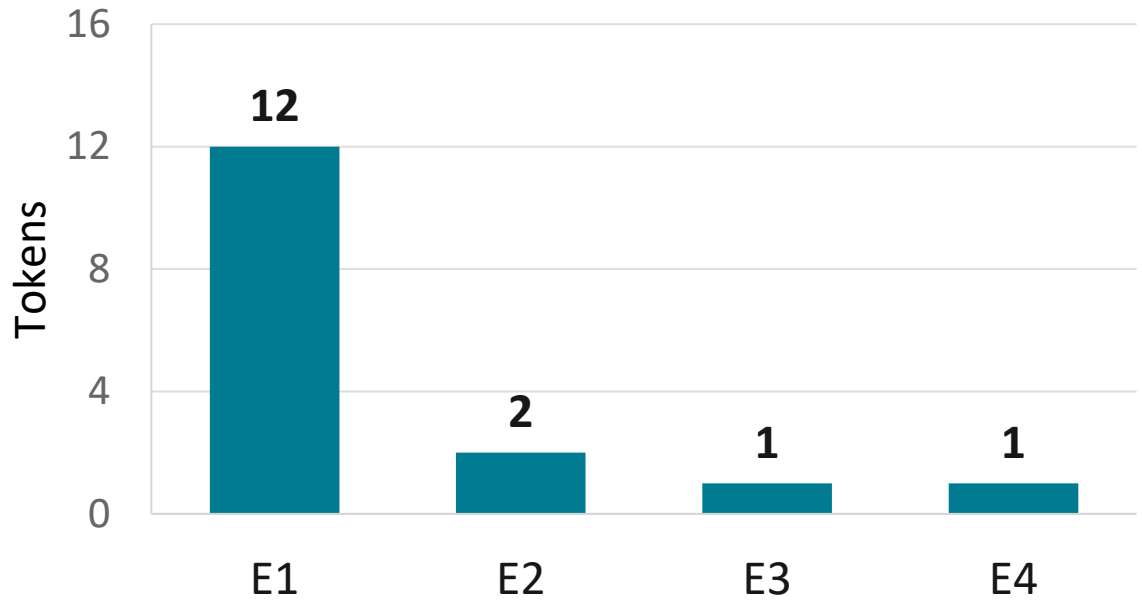
$$g_i = p_i \,/\, (\Sigma_{j \in I_k} p_j)$$

$$y = \Sigma_{i \in I_k} g_i \, E_i(x)$$

Shazeer et al., 2017; Dai et al., 2024

Uneven expert load

Sixteen tokens choose four experts with equal compute capacity.



An overloaded expert can delay the batch.

Other experts may sit idle and receive fewer training updates.

Balancing encourages more even use. Capacity handling may queue, drop or reroute tokens.

Fedus et al., 2021. Switch Transformers

An auxiliary loss for load balancing

For top-1 Switch routing, penalize experts that receive both high probability and many assignments.

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i$$

f_i: fraction of tokens assigned to expert i
P_i: mean router probability for expert i

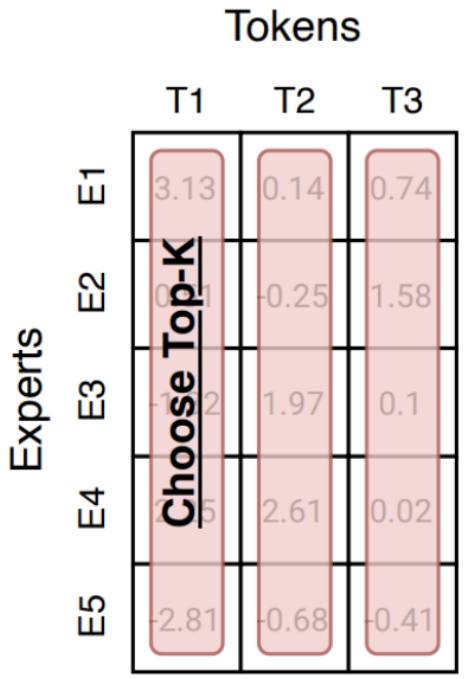
N: number of experts
α: strength of the auxiliary loss

Add this term to the language-model training loss.

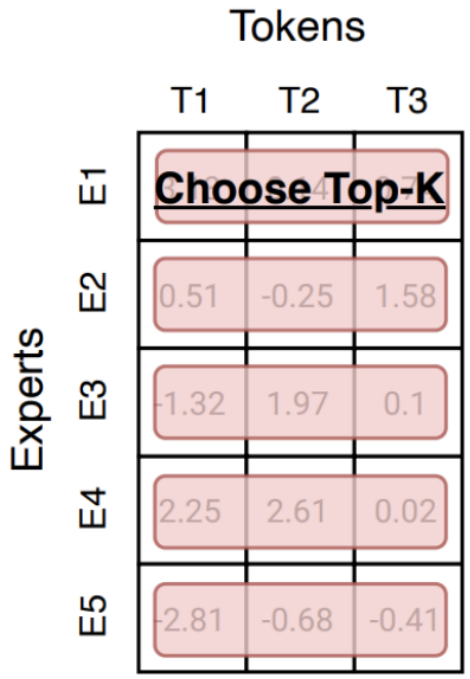
Fedus et al., 2021. Switch Transformers

Expert-routing alternatives

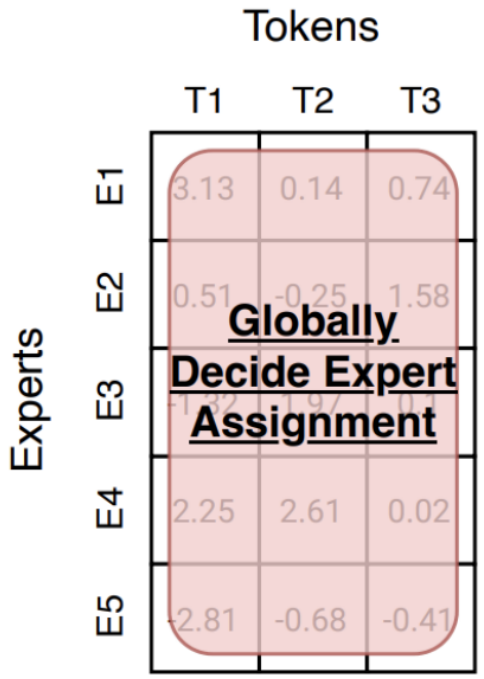
Routing alternatives change which side controls the assignment.



Token chooses expert



Expert chooses token



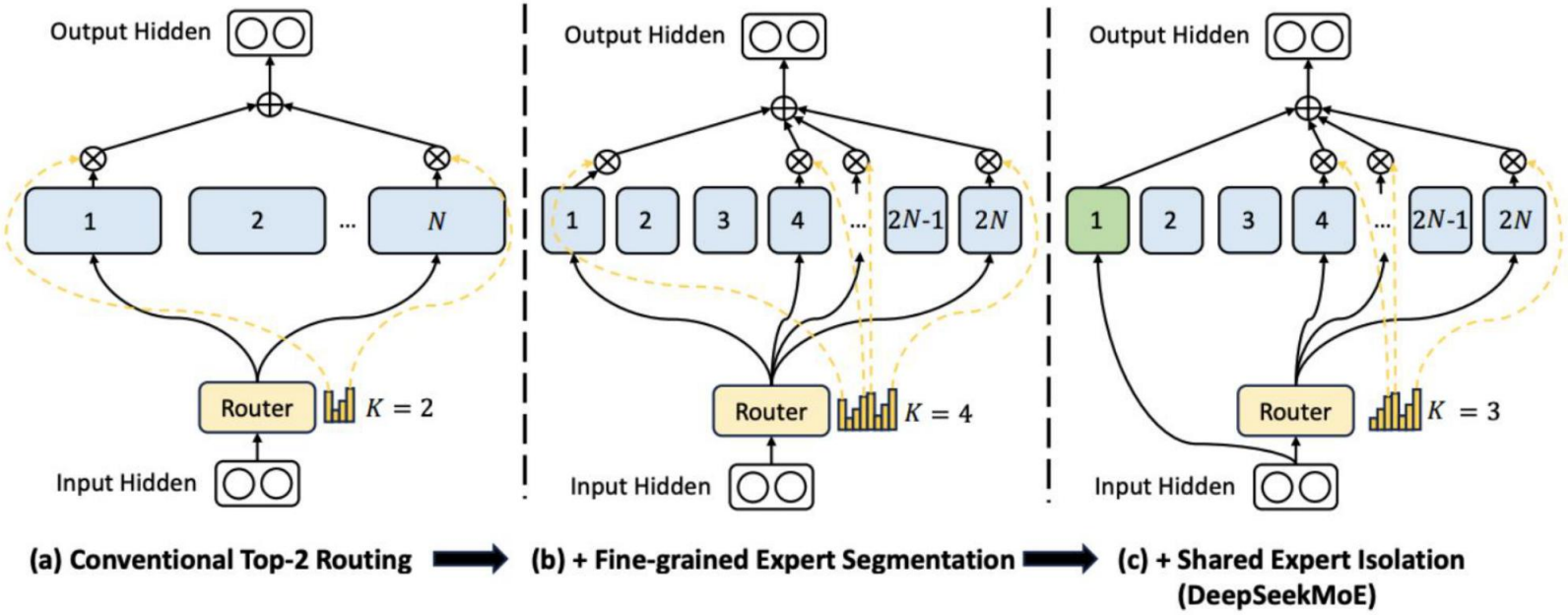
Global routing via optimization

[Fedus et al 2022]

Shared and fine-grained experts

Smaller routed experts provide more combinations at a fixed active budget.

Blue: routed experts. Green: shared experts.

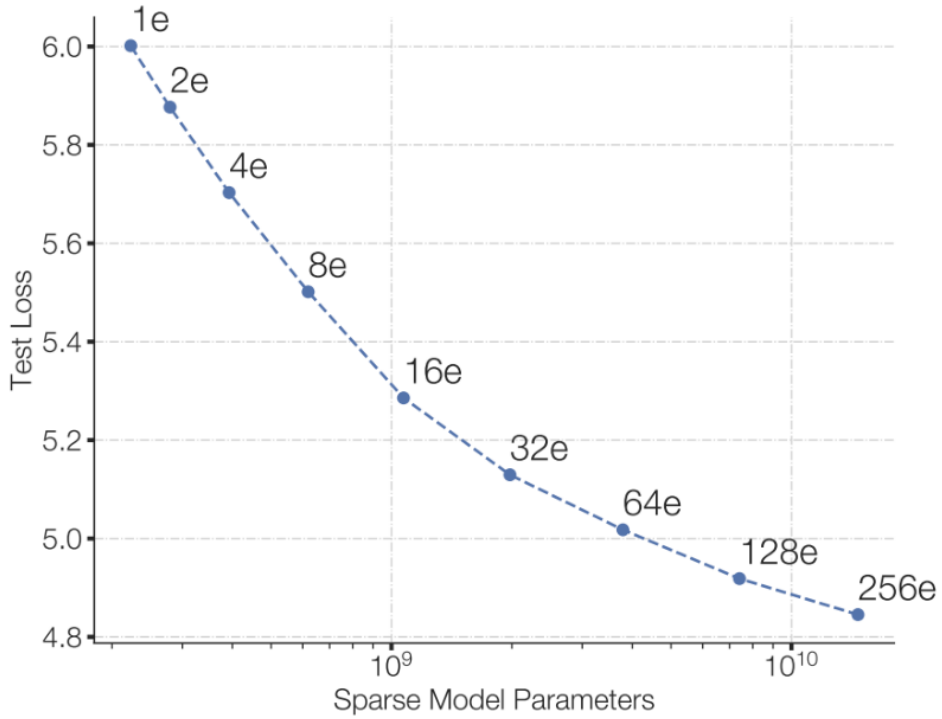


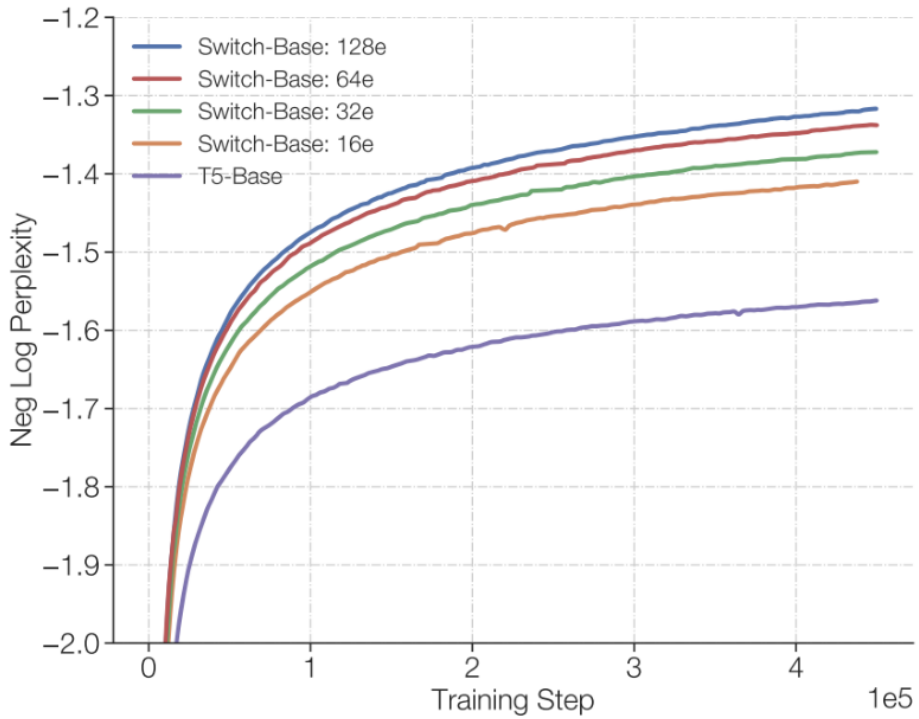
Shared experts run for every token and learn transformations used across inputs.

Dai et al., 2024. DeepSeekMoE

Switch Transformer: an empirical result

Switch Transformer experiments: more experts improve quality at matched per-token FLOPs.





[Fedus et al 2022]

Expert placement across devices

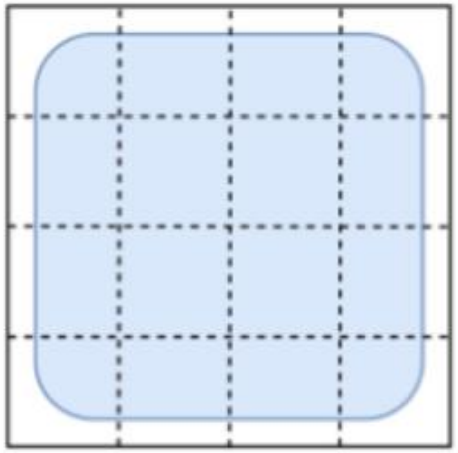
Expert parallelism stores different expert weights on different devices.

Weight placement



Each device owns a subset of experts.

Input batch



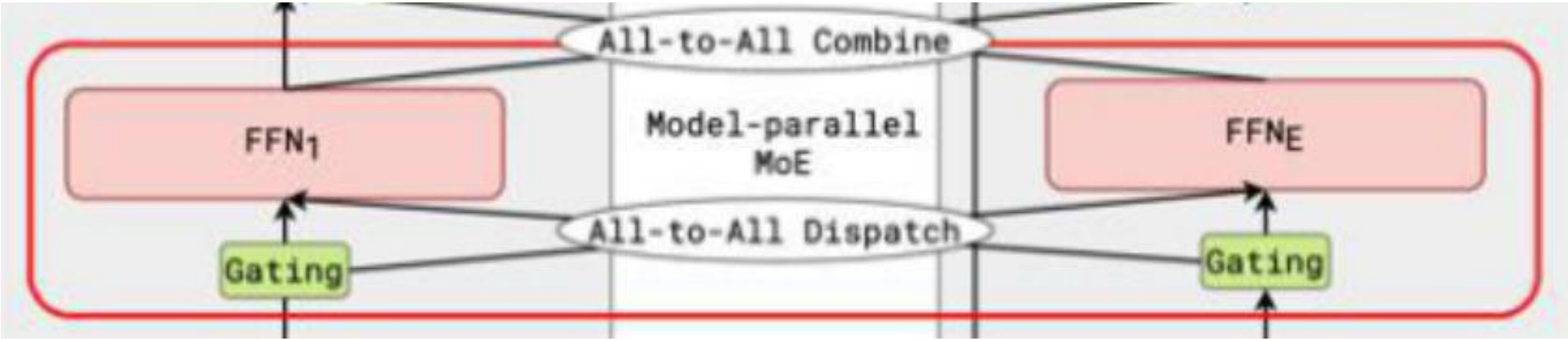
The token batch uses experts across devices.

Fedus et al., 2021. Switch Transformers

Dispatch, expert computation and combine

Experts on different devices exchange token activations.

Device 1 **Other expert devices**



1. Dispatch

Send each token to its selected experts.

2. Compute

Run each expert FFN on the tokens it receives.

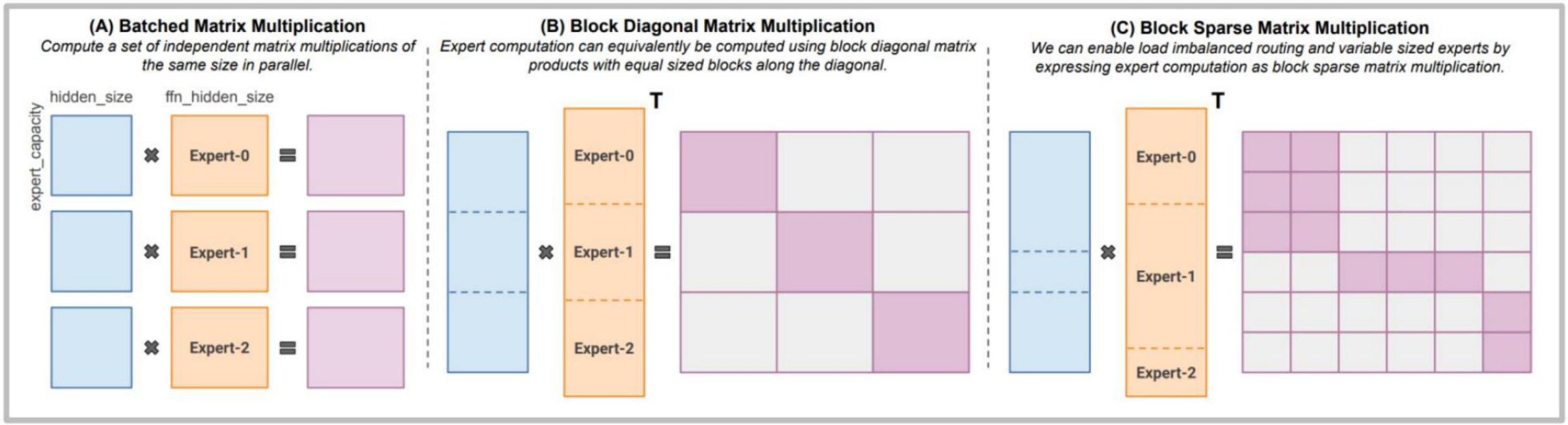
3. Combine

Return outputs and form the weighted sum.

Lepikhin et al., 2020. GShard

Efficient expert computation

Routing creates token groups of different sizes for each expert.



Grouped and block-sparse kernels handle uneven work without padding every expert to the same size.

Gale et al., 2023. MegaBlocks

The MoE trade-off

Total capacity

Many expert parameters remain available, even though only a few experts run per token.

Active computation

Top-k routing limits the expert FFNs used by each token.

Memory and communication

All weights still need storage. Routing adds token transfers and load-balancing constraints.

From architecture to execution cost

Block design

Normalization, residual paths and gating change training behavior and activation traffic.

Attention

Positions determine how order enters the model. K/V sharing and windows change memory and attention work.

Experts

Sparse computation adds capacity, while placement and routing determine communication costs.